

Tema 4



HTTP

1. Estructura de la WWW



- **World Wide Web:** conjunto de documentos enlazados unos con otros y escritos en hipertexto.
- **Hipertexto:** documento en el que puede aparecer texto, elementos multimedia, programas y vínculos o links hacia otros textos.
- **HTML (Hyper Text Mark-up Language):** lenguaje para diseñar páginas web basado en marcas o etiquetas. Define la estructura y contenido de los documentos de hipertexto.

1. Estructura de la WWW



- **HTTP (Hyper Text Transfer Protocol):** protocolo de transferencia de hipertexto. Empleado en las peticiones y respuestas de documentos web a través de la red. Sirve para la comunicación entre el navegador y el servidor.
- **Cliente web (navegador):** programa que lanza peticiones HTTP y muestra las webs obtenidas por pantalla, interpretando el HTML.
- **Servidor web:** host donde están alojados los documentos y páginas web. Tiene instalado un programa que responde las peticiones de los clientes.

2. URL



- Uniform Resource Locator
- RFC 1738
- Sirve para identificar cada uno de los recursos disponibles en la red (una foto de una página web, el fichero html de una página web, un fichero en un servidor FTP...)
- Se emplea sobre todo en HTTP
- Es la dirección que se escribe en el navegador

2. URL



- Formato básico:

- Protocolo://máquina/ruta/fichero

- Ejemplo:

- <http://www.prueba.es/ficheros/ejemplo/foto1.jpg>

- Protocolo: http

- Máquina: www.prueba.es

- Ruta: /ficheros/ejemplo

- Fichero: foto1.jpg

- En 'protocolo' y 'máquina', no se distingue entre mayúsculas y minúsculas. En el resto, depende del servidor y del SO

2. URL



- Otros ejemplos:

- <http://www.ejemplo.com/marzo/noticias/actualidad.html>
- <http://www.periodico.es/deportes/index.html>
- <http://www.descargas.es/docs/ej31.pdf>
- <http://www.portal.net/novedades>
- <http://www.prueba.es>
- www.buscador.es

2. URL



- Estructura del formato
 - Protocolo: puede ser http, https, ftp, mailto, file, spotify, skype...
 - Máquina: nombre completo del equipo o su IP.
OBLIGATORIO
 - Ruta: localización del fichero dentro del servidor
 - Fichero: nombre completo y extensión del archivo
 - Valores por defecto:
 - protocolo → http
 - ruta → raíz
 - fichero → index.html
 - Las URL se emplean en el atributo HREF de la etiqueta A del lenguaje HTML
 - `<A HREF="URL">Enlace</A>`

2. URL



- **Formato extendido**

- Puede incluirse el número de puerto detrás del servidor (por defecto, 80)
- <http://www.ejemplo.es:8080/blabla/etc.html>
- En el caso de FTP, podemos incluir el usuario y la contraseña en la URL
- `ftp://usu3:awer45@ftp.ejemplo.es/progs/aplicacion.exe`
- El fichero final puede ser un programa que necesite datos de entrada (por ejemplo, información tecleada por el usuario en una web)

2. URL



- Formato extendido

A screenshot of a web form. It contains four input fields: 'Nombre:' with the value 'Pepe', 'Apellidos:' with the value 'García Pérez', 'Edad:' with the value '37', and 'Estado civil:' with a dropdown menu showing 'Soltero'. Below the fields are two buttons: 'Enviar consulta' and 'Restablecer'.

- Al hacer clic en el botón Enviar se invoca a:

- <http://www.servidor.com/programas/prog1.php?nom=pepe&ape=Garcia+Perez&edad=37&ec=1>

- Formato general:

- <http://máquina/ruta/prog?par1=val1&par2=val2&par3=val3>

- <http://www.youtube.com/watch?v=3493SA230L>

3. HTTP



- Protocolo de nivel de aplicación para la transferencia de páginas web
- TCP, puerto 80
- RFC 1945, 2616, 2774
- Basado en mensajes solicitud/respuesta en formato ASCII
- Funcionamiento:
 - El cliente inicia una petición, estableciendo una conexión TCP con el puerto 80 del servidor
 - El servidor HTTP recibe el mensaje, procesa la solicitud y devuelve la respuesta
 - El navegador del cliente muestra la página web formateada

3. HTTP



- Es un protocolo sin estado”, es decir, no recuerda nada relativo a conexiones anteriores.
- La conexión sólo dura el tiempo suficiente para transmitir el fichero necesario, liberándose al finalizar cada petición.
- Por ejemplo, si una web contiene 3 imágenes, se lanzarán 4 peticiones HTTP: primero una para el fichero .html, y después una para cada una de las imágenes.

3. HTTP



- Comandos HTTP

- GET: obtener un recurso del servidor (por ejemplo, una página web)
- POST: enviar información al servidor (por ejemplo, los datos introducidos en un formulario)
- Las respuestas incluyen un código numérico y el contenido del fichero solicitado

3. HTTP



- Comando GET
 - Formato: GET fichero versiónHTTP
 - fichero es el archivo solicitado
 - versiónHTTP es la versión del protocolo que se empleará
 - Después del mensaje GET hay una serie de cabeceras que indican cierta información sobre el cliente:
 - User-Agent: navegador del cliente
 - Accept-Language: idiomas que se aceptan como respuesta
 - Host: nombre del servidor
 - Accept: tipos MIME aceptados por el cliente
 - A continuación viene una línea en blanco y el cuerpo del mensaje

3. HTTP



- Comando GET

- Ejemplo:

- GET /actualidad/noticias/ejemplo.html HTTP/1.1
 - User-Agent: Chrome/8.0.552.327
 - Accept-Language: es-ES
 - Host: www.periodico.es
 - Accept: */*
 - (línea en blanco)

3. HTTP



- Respuestas HTTP

- Primera línea: versiónHTTP código texto

- Códigos:

- 1xx: mensajes informativos
 - 2xx: operación realizada con éxito
 - 3XX: redirección a otra URL
 - 4xx: error en el cliente
 - 5xx: error en el servidor
 - A continuación vienen las cabeceras del servidor
 - Server: programa servidor utilizado
 - Date: fecha de envío
 - Content-Type: tipo MIME del fichero enviado
 - Content-Length: tamaño del fichero enviado
 - Last-Modified: fecha de la última modificación del fichero
 - Finalmente , tras una línea en blanco, se encuentra el contenido del fichero solicitado

3. HTTP



● Respuestas HTTP

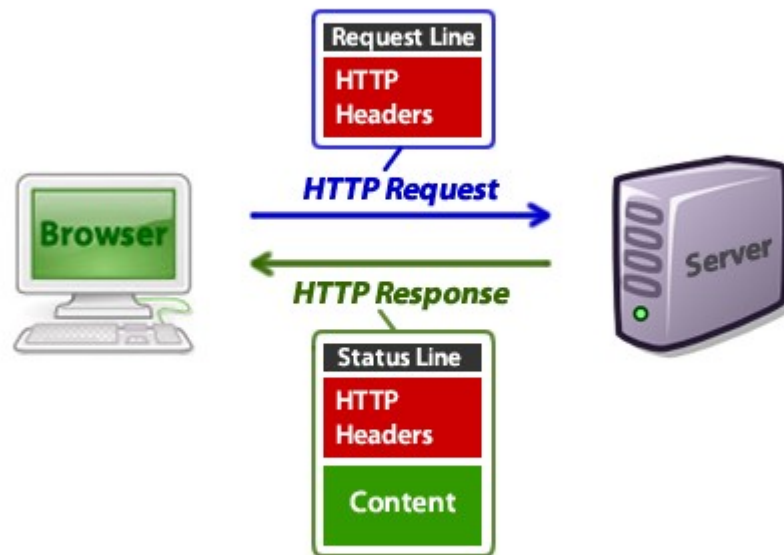
○ Ejemplo:

- HTTP/1.1 200 OK
- Date: Mon, 23 May 2010 14:35:52 GMT
- Server: Apache
- Last-Modified: Fri, 15 May 2010 10:40:23 GMT
- Content-Type: text/html
- Content-Length: 382
- (línea en blanco)
- <html>
- ...
- </html>

3. HTTP



- Diálogo HTTP



3. HTTP



- Comando GET (continuación)
 - También puede ser utilizado para enviar datos al servidor
 - Ejemplo:
 - GET /procesar.php?nom=Juan&edad=25 HTTP/1.1
 - Host: www.ejemplo.es
 - User-Agent: Mozilla/4.0
 - Limitaciones del método GET
 - Los datos pueden verse en la URL
 - La URL tiene un tamaño limitado, por lo que el envío de una gran cantidad de datos no es posible empleando este método

3. HTTP



- Comando POST

- Utilizado para enviar datos al servidor
- POST /procesar.php HTTP/1.1
- Host: www.ejemplo.es
- Content-Type: application/x-www-form-urlencoded
- Content-Length: 16
- (línea en blanco)
- nom=Juan&edad=25

- En el formulario debe aparecer el método de envío

- `<form name=registro action=procesar.php method=post>`
- `<form name=registro action=procesar.php method=get>`

3. HTTP



- Tipos MIME en HTTP

- Tienen tres funcionalidades
- Informar al cliente del tipo de datos que recibe
- Si es text/html (o tipos conocidos), el navegador lo visualiza
- Si es algún tipo que requiere alguna aplicación específica, la abre (visor de PDF, etc)
- Si es algún tipo que no conoce ,pregunta al usuario
- Permitir la negociación del contenido
- El cliente puede enviar en sus cabeceras los tipos que aceptará
- Encapsular varias partes en el cuerpo de un mensaje
- Típicamente usado cuando el cliente envía datos al servidor (formularios)

4. Tecnologías web



- Conjunto de tecnologías que aumentan la funcionalidad del HTML
 - CGI
 - Java Script
 - Cookies
 - Java
 - PHP
 - CSS
 - XHTML
 - HTML 5

4. Tecnologías web



- CGI

- Common Gateway Interface
- Permite que desde un cliente se envíen datos de un formulario a un programa ejecutable residente en el servidor
- Etiqueta `<form>`:
 - `<form method=get action=ejemplo.cgi>...</form>`
- El programa CGI recibe los datos del cliente enviados mediante el método GET o POST, los procesa, y devuelve el resultado en forma de respuesta HTTP, que incluye el código de la página web generada
- El CGI se activa al hacer clic en el botón de "Submit"

4. Tecnologías web



- CGI

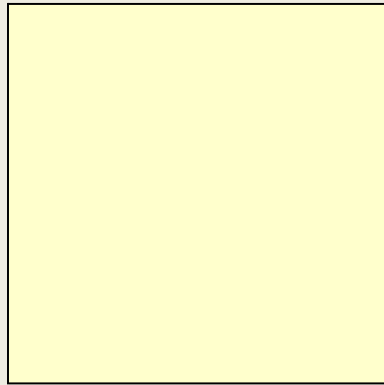
- Lenguajes de programación para escribir CGI: Perl, C, shell scripts escritos en el lenguaje de comandos de Linux...
- La ejecución del CGI se realiza en el servidor
- La página web generada es devuelta al cliente y mostrada en su navegador
- Dicha página web es dinámica (depende de los datos introducidos en el formulario) y volátil (no se almacena como fichero en el servidor)

4. Tecnologías web

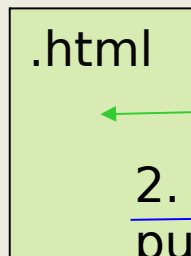
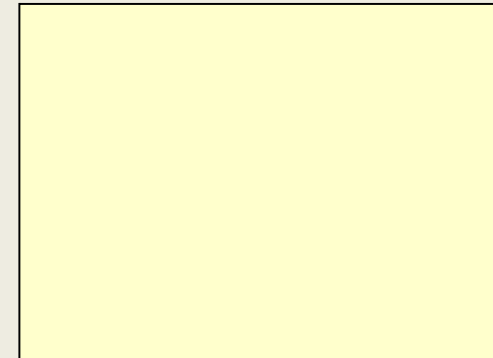


• CGI

Cliente



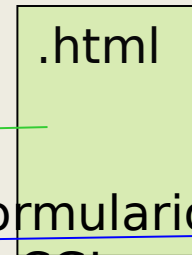
Servidor



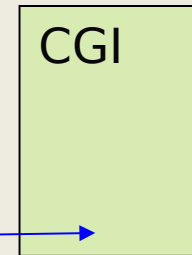
.html

1. El formulario
viaja al cliente

2. El cliente rellena el formulario,
pulsa Enviar e invoca al CGI



.html



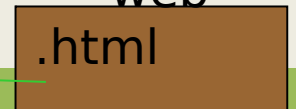
CGI

3. El
CGI
genera
una
web



.html

4. La web generada dinámicamente viaja al cliente



.html

4. Tecnologías web



- CGI

- Ejemplo:

- Fichero formulario.html

- ...

- `<form method=get action=/cgi-bin/ejemplo.cgi>`

- Introduce tu nombre:

- `<input type=text name=nombre><br>`

- Introduce tu edad:

- `<input type=text name=edad><br>`

- `<input type=submit name=env value=Enviar>`

- `<input type=reset name=bor value=Borrar>`

- `</form>`

- ...

4. Tecnologías web



● CGI

- Ejemplo:
- Fichero ejemplo.cgi
 - `#!/bin/bash`
 - `echo "Content-type: text/html"`
 - `echo ""`
 - `echo "<html><head><title>Respuesta</title></head>"`
 - `echo "<body>"`
 - `n=`echo $QUERY_STRING | cut -d'&' -f1 | cut -d'=' -f2``
 - `e=`echo $QUERY_STRING | cut -d'&' -f2 | cut -d'=' -f2``
 - `if [ $e -ge 18 ] ; then`
 - `echo "Bienvenido <b> $n </b>"`
 - `else`
 - `echo "<font color=red> No puedes entrar </font>"`
 - `fi`
 - `echo "</body></html>"`
 - `QUERY_STRING → "nombre=Alberto&edad=35"`

4. Tecnologías web



- CGI

- Ventajas

- Mayor dinamismo e interacción
 - Páginas web generadas dependiendo de los datos del cliente
 - Multitud de lenguajes para realizar CGI

- Inconvenientes

- Desde el cliente, estamos ejecutando un programa que reside en el servidor (posibles fallos de seguridad)
 - Consumo de recursos en el servidor
 - Las tareas que realiza directamente el CGI no pueden ser muy complejas

4. Tecnologías web



- **Java Script**

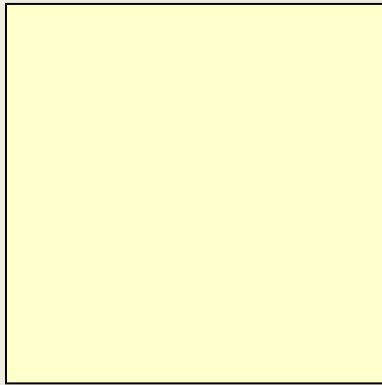
- Lenguaje de programación
- Útil para páginas web
- Sintaxis parecida a Java
- El código de los programas va incrustado en el interior de la página web, encerrado entre las etiquetas `<script>` y `</script>`
- El código JS se ejecuta en el cliente
- Útil para comprobar los valores de los campos de un formulario y para pequeñas aplicaciones

4. Tecnologías web

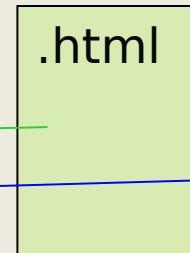
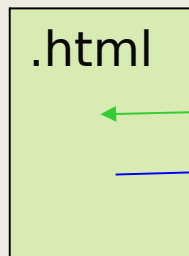
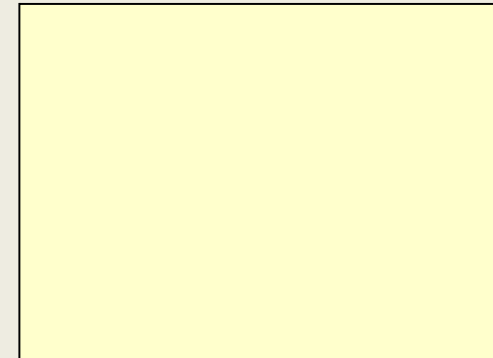


• Java Script

Cliente



Servidor



1. El formulario
viaja al cliente

3. Sólo si los
datos pasan el
test, se envían

2. Al pulsar el botón de Enviar, se
ejecuta código JS para validar datos

4. Tecnologías web



- Java Script

- Ejemplo:

- Fichero formulario.html
 - ...
 - `<script>`
 - `function comprobar(){`
 - `if (formulario.edad.value>100) {`
 - `alert("La edad introducida es demasiado alta");`
 - `return false;`
 - `} else return true;`
 - `}`
 - `</script>`
 - ...
 - `<form name=formulario method=get action=/cgi-bin/ejemplo.cgi>`
 - Introduce tu nombre:
 - `<input type=text name=nombre><br>`
 - Introduce tu edad:
 - `<input type=text name=edad><br>`
 - `<input type=submit name=env value=Enviar onClick="comprobar()">`
 - `<input type=reset name=bor value=Borrar>`
 - `</form>`
 - ...

4. Tecnologías web



● Java Script

○ Ventajas

- Ejecución en el cliente (menos carga en el servidor)
- Potente lenguaje para validación de datos, útil en el paso previo al envío→libera de trabajo (detección de errores) al CGI
- Código incrustado en el HTML
- Gestión de eventos

○ Inconvenientes

- Limitación de los scripts (no pueden acceder a Bases de Datos...)

4. Tecnologías web



- Cookies

- Ficheros de texto almacenados en el cliente
- En Windows están en C:\Documents and Settings*nombreusuario*\Cookies
- Contenido de una cookie: información útil y frecuentemente usada, que debe residir en el cliente
- Se crean una vez (la primera ocasión que el usuario accede a una web concreta) y se leen el resto de veces
- Ejemplo de lo que almacena una cookie: idioma preferido de una web, resolución, nombres y contraseñas, personalización de una página...
- Por lo general, el contenido de una cookie está encriptado
- Lenguajes para el manejo de cookies: JS, PHP...

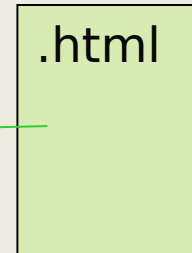
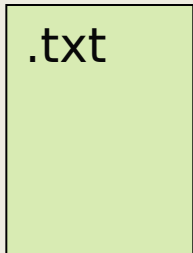
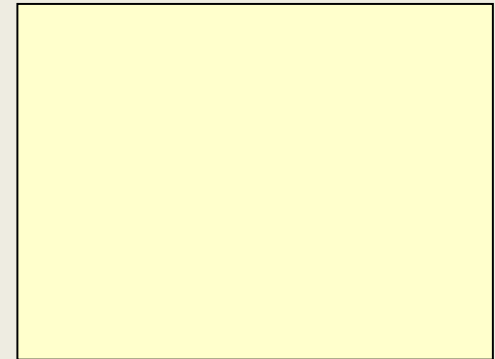
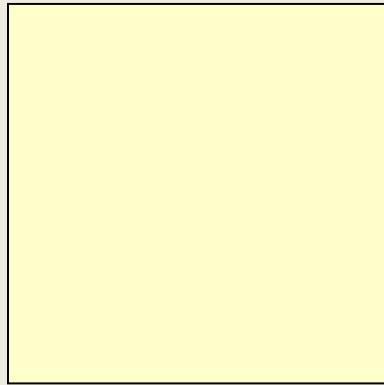
4. Tecnologías web



- Cookies

Cliente

Servidor



1. El formulario
viaja al cliente



2. En el cliente, se genera y guarda la cookie. Posteriores accesos a la web leerán la cookie

4. Tecnologías web



- Cookies

- Ventajas

- Al ser ficheros de texto, no existe riesgo de seguridad.
 - Permiten recordar pequeños valores, útiles para no repetirlos constantemente.

- Inconvenientes

- Solo pueden almacenar valores, no son programas.
 - Ocupan espacio en el cliente, algunas acumulándose innecesariamente.
 - Difícil comprensión del contenido por parte del usuario.

4. Tecnologías web



● Java

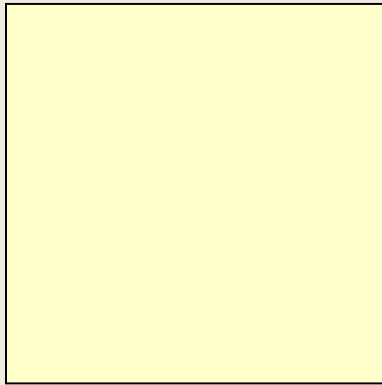
- Lenguaje de programación de propósito general creado por Sun Microsystems.
- Diferencias con JS: Java se compila y el código está en un fichero aparte, mientras que JS se interpreta y el código va incrustado en el mismo fichero .html.
- El programa Java ("applet") reside en el servidor, y viaja hacia el cliente.
- Se ejecuta en el interior del navegador del cliente, en la JVM (Java Virtual Machine).
- Java no es exclusivo para webs, tiene otros muchos usos.
- Etiqueta applet:
 - `<applet codebase=programa.class width=400 height=200>`

4. Tecnologías web

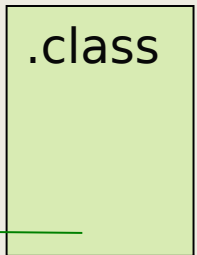
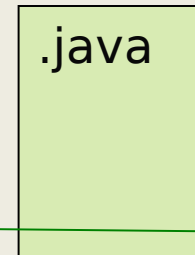
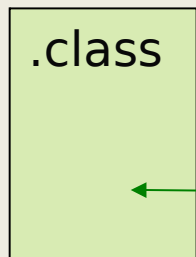
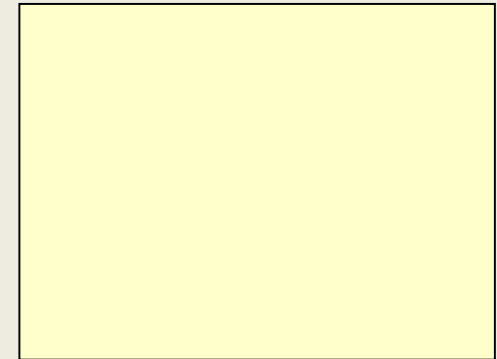


• Java

Cliente



Servidor



1. La web viaja al cliente

2. El fichero con el programa Java también viaja al cliente

Código
fuente

Programa
Java

3. El código Java se ejecuta en el navegador

4. Tecnologías web



● Java

○ Ejemplo programa.java

- **import** java.applet.*;
- **import** java.awt.*;
- **public class** HelloWorldApplet **extends** **Applet** {
- **public** void init() { }
- **public** void start() { }
- **public** void stop() { }
- **public** void destroy() { }
- **public** void paint(**Graphics** g) {
- g.setColor(**Color**.GREEN);
- g.drawString("Hello World", 50, 100);
- }
- }

4. Tecnologías web



- Java

- Ventajas

- Potente, general, orientado a objetos
 - Independiente de la plataforma

- Inconvenientes

- Lentitud debido a los niveles de traducción (navegador+SO+hardware)
 - Alto consumo de recursos

4. Tecnologías web



● PHP

- Lenguaje de programación
- Especialmente útil en combinación con SQL (otro lenguaje que permite obtener información de una Base de Datos)
- Empleado cuando hay que acceder a una gran cantidad de información de manera muy organizada (ej: foros, noticias de portales...)
- El código PHP va dentro del fichero HTML entre las etiquetas `<?php` y `?>`
- Los ficheros .html que tienen código PHP en su interior reciben la extensión .php
- El servidor necesita tener instalado un servidor web + un intérprete de PHP + un servidor de SQL
- Ejemplo: WAMP (Windows+Apache+MySQL+PHP)

4. Tecnologías web



● PHP: cliente

- 1. El usuario rellena un formulario de una web
- 2. El usuario hace clic en el botón de Enviar
- 3. Los datos se envían al servidor
- 8. El cliente recibe la web

● PHP: servidor

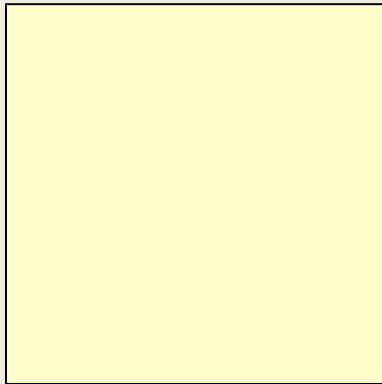
- 4. El servidor ejecuta el código PHP
- 5. El programa PHP accede a una BD mediante instrucciones escritas en el lenguaje SQL
- 6. El programa PHP obtiene los datos de la BD y genera una página web de respuesta
- 7. El servidor devuelve la página generada al cliente

4. Tecnologías web



● PHP

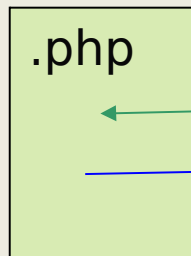
Cliente



Servidor

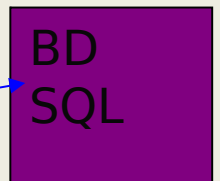
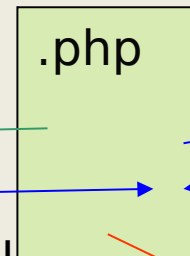


2. El cliente rellena el formulario y pulsa Enviar



1. El formulario viaja al cliente

3. Los datos viajan al servidor



4. El servidor consulta la BD y genera una web

.html

5. La web viaja al cliente

.html



4. Tecnologías web

- PHP

- Ejemplo : fichero buscador.php



- **<html><body>**
 - **<?php**
 - **if (isset(\$_POST["buscar"])) {**
 - **\$link = mysql_connect("localhost", "root", "");**
 - **mysql_select_db("bdo", \$link);**
 - **\$nombre = \$_POST["nompsona"];**
 - **\$consulta = "SELECT * FROM personas WHERE nom like '\$nombre';"**
 - **\$result = mysql_query(\$consulta, \$link);**
 - **echo "<table border=1>";**
 - **while(\$row = mysql_fetch_array(\$result))**
 - **echo "<tr><td>".\$row["nom"]."</td><td>".\$row["ape"]."</td></tr>";**
 - **mysql_close(\$link);**
 - **echo "</table>";**
 - **}else{**
 - **?>**
 - **<form method="post" action="buscador.php">**
 - **Nombre a buscar: <input type="text" name="nompsona" size="20">
**
 - **<input type="submit" name="buscar" value="Comenzar búsqueda">**
 - **</form>**
 - **<?php**
 - **}**
 - **?>**
 - **</body></html>**
 - **Código HTML, Código PHP, Código SQL**

4. Tecnologías web



- PHP

- Ventajas

- Potentísimo
 - Velocidad: el PHP no se compila, se interpreta
 - Actualmente: combinación HTML (presentación) + JS(validación)+PHP(generación respuesta)+SQL(Base de Datos)

- Inconvenientes

- Conocimientos de Bases de Datos
 - Conocimientos de SQL
 - Servidor sobrecargado: servidor SQL + intérprete PHP + servidor Web
 - Espacio para los .html y para las BD (que pueden ocupar TB)

4. Tecnologías web



- CSS

- Cascading Style Sheets (Hojas de estilo)
- Permiten aplicar un mismo aspecto a diferentes webs
- Se define, para cada etiqueta HTML, qué propiedades va a tener, y se guardan en una hoja de estilo
- A cada web se le aplica la hoja de estilo
- De este modo, no hace falta especificar las mismas propiedades (color, fondo, etc) varias veces en numerosas páginas
- Separan la estructura y la presentación de un documento

4. Tecnologías web



- CSS

- Ejemplo sin hojas de estilo:

- fich1.html

- `<html><head></head>`

- `<body bgcolor=blue>`

- `<h1><font face="Courier" color=yellow>Esto es un ejemplo</font></h1>`

- `</body></html>`

- fich2.html

- `<html><head></head>`

- `<body bgcolor=blue>`

- `<h1><font face="Courier" color=yellow>Y esto es otro ejemplo</font></h1>`

- `</body></html>`

4. Tecnologías web



● CSS

- Ejemplo con hojas de estilo:

- fich1.html

- `<html><head><link rel="stylesheet" type="text/css" href="miestilo.css"></head>`

- `<body>`

- `<h1>Esto es un ejemplo</h1>`

- `</body></html>`

- fich2.html

- `<html><head><link rel="stylesheet" type="text/css" href="miestilo.css"></head>`

- `<body>`

- `<h1>Y esto es otro ejemplo</h1>`

- `</body></html>`

4. Tecnologías web



- CSS

- Ejemplo con hojas de estilo:

- miestilo.css

- body {

- background-color: blue

- }

- h1 {

- font-family: Courier;

- color:yellow

- }

- En las hojas de estilo aparecen tres elementos: selector (body), propiedad (background-color) y valor (blue)

4. Tecnologías web



- CSS

- Ventajas

- Gestión centralizada del estilo de un sitio web completo (basta con modificar el fichero .css para que todos los .html que hagan referencia a él cambien su aspecto a la vez)
 - Fácil cambio de estilo (basta con sustituir el fichero .css por otro)

- Inconvenientes

- Más ficheros en el servidor (.css)
 - Sintaxis especial en el interior del .css

4. Tecnologías web



- XHTML

- eXtensible HyperText Markup Language (Lenguaje extensible de marcas de hipertexto)
- XHTML es la evolución de HTML para que cumpla la normativa XML
- XML fue creado no solo para webs, sino en general para cuando los datos tienen cierta estructura
- XML se usa para estructurar documentos en general, no tienen por qué estar relacionados con Internet
- XML permitiría crear nuevas etiquetas por parte del usuario: <ejemplo></ejemplo>, <persona></persona>
- Con XHTML, los datos de una página se separan de su presentación en diversos dispositivos (ordenadores, móviles, PDA...)

4. Tecnologías web



● XHTML

○ Diferencias entre HTML y XHTML

- Todas las etiquetas deben cerrarse siempre: `<br></br>` (o más abreviadamente `<br/>` o también `<br />`)
- Los elementos anidados deben cerrarse siempre en orden inverso: `<b><u>...</u></b>`
- Los atributos deben llevar siempre comillas (simples o dobles): `<td colspan="2">` o `<td colspan='2'>`
- Las imágenes deben tener textos alternativos siempre: ``
- Los elementos y atributos siempre en minúsculas: `<a href="http://www.google.com"> ... </a>`
- Los atributos siempre han de tener valor: `<input type="checkbox" name="ejemplo" checked="yes" />`

4. Tecnologías web



- XHTML

- Ventajas

- Un mismo documento puede tener apariencias distintas en diferentes dispositivos
 - Ayuda a la estandarización del XML

- Inconvenientes

- Muchos navegadores (Internet Explorer) no cumplen la normativa XHTML (pese a que Microsoft forma parte del equipo de desarrollo de XML)
 - Tradición del HTML
 - Hay entornos de diseño de páginas web que no generan el código XHTML correcto

4. Tecnologías web



● HTML 5

- Revisión del HTML para acoger los cambios y las necesidades de las webs actuales
- Nuevas etiquetas: `<audio>`, `<video>`, `<article>`...
- Incluye mejoras para validar el contenido sin JS
- Nuevos tipos: `month`, `date`, `time`, `email`, `tel`, `color`...
- Nueva estructura de las páginas: cabecera, cuerpo, pie, barras laterales...
- Manejo de BD locales para aplicaciones web
- Geolocalización
- Eliminación de todas las etiquetas de presentación

5. HTTPS



- Versión segura de HTTP
- Puerto 443
- Puede utilizar dos protocolos criptográficos para garantizar la seguridad
 - SSL 3.0 (Secure Sockets Layer)
 - TLS 1.0 (Transport Layer Security): evolución de SSL
- Empleado para el envío de contraseñas, pagos con tarjeta...

5. HTTPS



- Requiere que se confíe en la Autoridad de Certificación
 - Entidad que garantiza la validez de una clave pública
 - Permite asociar una clave pública con una entidad
 - La clave pública de la entidad se envía con el certificado
 - Junto con la clave privada, permitirá autenticar clientes y servidores
 - Nos indica en qué entidades podemos confiar plenamente

5. HTTPS



- Basado en
 - Encriptación: cifrar lo que se envía de un equipo a otro.
 - Identificación: asegurar que podemos confiar en el equipo al que nos conectamos.
- Autenticación del servidor
 - Permite que los clientes envíen datos a un servidor que tenga un Certificado de Servidor.
 - El administrador de un portal que quiera ser seguro debe pedir un Certificado a una Autoridad de Certificación e instalarlo en el servidor (certificados públicos) o crear un Certificado por su cuenta (certificados privados).

5. HTTPS



- Autenticación del cliente

Permite que los clientes tengan acceso a áreas protegidas del servidor. Los clientes deberán tener instalado el Certificado de Cliente.

El servidor creará un Certificado de Cliente para cada usuario, que se guardará en el navegador y se revisará en cada conexión.

5. HTTPS



- Tipos de certificados

- Todos los certificados incluyen la clave pública correspondiente, la firma digital de la Autoridad de Certificación y la fecha de expiración
- Públicos
 - Realizados por las AC típicas (VeriSign, ACCV...)
 - Son de pago (aprox. 200 €/año)
 - No requieren ningún tipo de instalación en los clientes
- Privados
 - Gratuitos, creados por la propia empresa que gestiona la web
 - Cuando los clientes se conectan a un servidor con un certificado de este tipo, se les pregunta si confían en ese certificado.